

2025 z.

# РЕФЕРАТ

Расчетно-пояснительная записка по курсовой работе на тему «Разработка программного решения для трехмерной визуализации и генерации городской среды» содержит 32 с., 10 рис., 1 табл., 16 ист, 2 прил.

Работа посвящена разработке программного комплекса для процедурной генерации и интерактивной 3D-визуализации городской среды. Цель достигнута через реализацию алгоритмов построения связной транспортной сети с множеством перекрёстков, создания моделей зданий, а также освещения на основе карты теней. Обеспечены воспроизводимость сцен и интерактивное управление камерой.

В основе решения — применение Z-буфера, диффузной закраски и аффинных преобразований. Приложение написано на C++20 и Qt6.5 и разделено на модули генерации, рендеринга и UI. Исследования показали квадратичную зависимость времени отрисовки от количества пикселей кадра.

Ключевые слова: Z-буфер, город, карта теней, C++20, Qt6.5.

# СОДЕРЖАНИЕ

|                                                                        |           |
|------------------------------------------------------------------------|-----------|
| <b>РЕФЕРАТ . . . . .</b>                                               | <b>4</b>  |
| <b>ВВЕДЕНИЕ . . . . .</b>                                              | <b>6</b>  |
| <b>1 Аналитическая часть . . . . .</b>                                 | <b>7</b>  |
| 1.1. Городская среда . . . . .                                         | 7         |
| 1.2. Аффинные преобразования для трехмерных моделей . . . . .          | 7         |
| 1.3. Методы представления трехмерных объектов . . . . .                | 9         |
| 1.4. Описание алгоритмов . . . . .                                     | 10        |
| 1.4.1. Методы проецирования трехмерных объектов на двумерную плоскость | 10        |
| 1.4.2. Методы удаления невидимых поверхностей . . . . .                | 11        |
| 1.4.3. Методы освещения и расчета теней . . . . .                      | 12        |
| 1.4.4. Выбранные методы для визуализации городской среды . . . . .     | 13        |
| <b>2 Конструкторская часть . . . . .</b>                               | <b>15</b> |
| 2.1. Схемы алгоритмов . . . . .                                        | 15        |
| 2.2. Используемые структуры данных . . . . .                           | 17        |
| <b>3 Технологическая часть . . . . .</b>                               | <b>19</b> |
| 3.1. Выбор технологий для программной реализации . . . . .             | 19        |
| 3.2. Структура классов . . . . .                                       | 19        |
| 3.3. Создание городской среды . . . . .                                | 19        |
| 3.4. Пример работы программы . . . . .                                 | 21        |
| <b>4 Исследовательская часть . . . . .</b>                             | <b>23</b> |
| 4.1. Результаты замеров . . . . .                                      | 23        |
| <b>ЗАКЛЮЧЕНИЕ . . . . .</b>                                            | <b>25</b> |
| <b>СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ . . . . .</b>                      | <b>26</b> |
| <b>ПРИЛОЖЕНИЕ А . . . . .</b>                                          | <b>28</b> |
| <b>ПРИЛОЖЕНИЕ Б . . . . .</b>                                          | <b>32</b> |

# ВВЕДЕНИЕ

Целью курсовой работы является разработка программного решения для генерации и трёхмерной визуализации городской среды.

- Разработать алгоритм генерации транспортной сети, представленной соединёнными отрезками дорог с не менее чем тремя перекрёстками и отсутствием изолированных участков.
- Создать объемные модели зданий с поддержкой не менее пяти архитектурных типов, обеспечивающую размещение объектов вдоль дорог без взаимных пересечений и минимальную детализацию двадцать полигонов на модель.
- Реализовать механизм воспроизводимой генерации городской среды с фиксацией начального значения генератора псевдослучайных чисел.
- Разработать модуль интерактивного управления виртуальной камерой, поддерживающий операции перемещения, вращения и масштабирования сцены в реальном времени.
- Интегрировать алгоритм карты теней для расчёта освещения с возможностью настройки параметров источников света.
- Провести анализ производительности решения, исследовав зависимость времени отрисовки кадра от разрешения изображения и количества полигонов в экранном пространстве.

# 1 Аналитическая часть

В данной части формализованы объекты сцены, а также рассмотрены основные методы решения поставленных задач.

## 1.1. Городская среда

Структура генерации городской среды основана на двух типах дорог: внешних, образующих границы кварталов, и внутренних, проходящих внутри кварталов. Вдоль внутренних дорог производится автоматическая расстановка зданий с учетом заданных параметров размера города и числа начального числа псевдослучайной последовательности.

Для визуализации городской среды используются следующие объемные модели.

- Дорога – связанный с другими набор полигонов, расположенных в горизонтальной плоскости. Проспекты – задают границы кварталов, внутриквартальные дороги образуют случайный набор дорог внутри квартала.
- Дом представляет собой набор полигонов, содержащий набор стен и крыши. Отдельно описан первый этаж, содержащий вход, типовой этаж, который одинаково описывает 2..N этажи, где N – количество этажей дома и крышу. Располагаются вдоль внутриквартальных дорог.
- Деревья – располагаемые вдоль внутриквартальных дорог объемные объекты, с горизонтальным сечением в виде правильного восьмиугольника.

## 1.2. Аффинные преобразования для трехмерных моделей

Аффинные преобразования являются фундаментальными операциями в компьютерной графике, позволяющими выполнять геометрические преобразования моделей с сохранением коллинеарности точек и отношения расстояний вдоль прямых линий. В общем виде аффинное преобразование можно представить как комбинацию линейного преобразования и переноса:

$$\mathbf{P}' = \mathbf{M} \cdot \mathbf{P} + \mathbf{T}, \quad (1.1)$$

где  $\mathbf{P}$  – исходная точка,  $\mathbf{P}'$  – преобразованная точка,  $\mathbf{M}$  – матрица линейного преобразования размером  $3 \times 3$ ,  $\mathbf{T}$  – вектор переноса.

Для удобства вычислений в компьютерной графике используется однородная система координат, что позволяет представить любое аффинное преобразование в виде единой

матрицы  $4 \times 4$ :

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} & t_x \\ a_{21} & a_{22} & a_{23} & t_y \\ a_{31} & a_{32} & a_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad (1.2)$$

где верхняя левая подматрица  $3 \times 3$  описывает линейное преобразование, а правый столбец задает вектор переноса.

Основными типами аффинных преобразований, применяемых к трехмерным моделям, являются:

## Перенос

Преобразование переноса смещает все точки модели на заданный вектор  $\mathbf{T} = (t_x, t_y, t_z)$ :

$$\mathbf{T}(t_x, t_y, t_z) = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad (1.3)$$

где  $t_x, t_y, t_z$  – компоненты вектора переноса вдоль соответствующих осей координат.

## Масштабирование

Преобразование масштабирования изменяет размеры модели относительно начала координат:

$$\mathbf{S}(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad (1.4)$$

где  $s_x, s_y, s_z$  – коэффициенты масштабирования по осям  $x, y$  и  $z$  соответственно.

## Вращение

Вращение модели вокруг оси  $x$  на угол  $\theta$  описывается матрицей:

$$\mathbf{R}_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (1.5)$$

Вращение вокруг оси  $y$ :

$$\mathbf{R}_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (1.6)$$

Вращение вокруг оси  $z$ :

$$\mathbf{R}_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad (1.7)$$

где  $\theta$  – угол поворота в радианах.

Сложные преобразования формируются путём композиции базовых аффинных преобразований. Поскольку матричное умножение не коммутативно, порядок применения преобразований существенно влияет на результат. Для преобразования точки  $\mathbf{P}$  с помощью последовательности преобразований  $\mathbf{A}_1, \mathbf{A}_2, \dots, \mathbf{A}_n$  используется формула:

$$\mathbf{P}' = \mathbf{A}_n \cdot \mathbf{A}_{n-1} \cdot \dots \cdot \mathbf{A}_1 \cdot \mathbf{P}, \quad (1.8)$$

где преобразования применяются в порядке справа налево.

Аффинные преобразования обладают важным свойством инвариантности относительно выпуклых комбинаций точек. Если точка  $\mathbf{Q}$  является выпуклой комбинацией точек  $\mathbf{P}_1$  и  $\mathbf{P}_2$ :

$$\mathbf{Q} = \alpha \cdot \mathbf{P}_1 + (1 - \alpha) \cdot \mathbf{P}_2, \quad 0 \leq \alpha \leq 1 \quad (1.9)$$

то после применения аффинного преобразования  $\mathbf{A}$  сохраняется соотношение:

$$\mathbf{A}(\mathbf{Q}) = \alpha \cdot \mathbf{A}(\mathbf{P}_1) + (1 - \alpha) \cdot \mathbf{A}(\mathbf{P}_2). \quad (1.10)$$

Это свойство гарантирует сохранение формы и пропорций модели при преобразованиях [1].

### 1.3. Методы представления трехмерных объектов

Существует несколько основных подходов к представлению трехмерных объектов в компьютерной графике. Первый метод – сплошное представление, при котором объект описывается аналитическими уравнениями или неявными функциями. Данный подход

обеспечивает высокую точность геометрического описания, но требует значительных вычислительных ресурсов для рендеринга.

Второй метод – представление на основе вокселей, где пространство разбивается на равномерную трехмерную сетку, и каждый элемент хранит информацию о принадлежности к объекту. Этот подход эффективен для объемных данных, но имеет высокую память для хранения информации. Третий метод – каркасное представление, в котором объект описывается набором вершин и соединяющих их рёбер. Данный метод позволяет быстро визуализировать геометрию, но не обеспечивает реалистичного отображения поверхностей.

Четвертый метод – полигональное представление, при котором поверхность объекта аппроксимируется набором плоских многоугольников. По соображениям вычислительной эффективности и простоты обработки в компьютерной графике преимущественно используются треугольные полигоны, поскольку любой треугольник всегда является плоским и легко поддается обработке. Математически полигональная модель объекта может быть представлена как множество вершин  $V = \{v_1, v_2, \dots, v_n\}$  и множество граней  $F = \{f_1, f_2, \dots, f_m\}$ , где каждая грань  $f_i$  определяется набором индексов вершин [2]. Полигональное представление также можно дополнить при помощи указания нормалей поверхности, это позволяет отсекал поверхности выпуклых объемных моделей, которые повернуты от направления наблюдателя, и, соответственно не могут быть видны с текущего ракурса.

В рамках визуализации городской среды полигональное представление с треугольными полигонами является оптимальным выбором, поскольку позволяет эффективно моделировать сложные архитектурные формы зданий и инфраструктуры при приемлемых вычислительных затратах, также будет использовано предвычисление нормалей полигонов для обеспечения отсечений.

## **1.4. Описание алгоритмов**

### **1.4.1. Методы проецирования трехмерных объектов на двумерную плоскость**

Для отображения трехмерных объектов на двумерном экране используются различные методы проецирования. Ортогональная проекция сохраняет параллельность линий и пропорции объектов, но не создает эффекта глубины. Математически ортогональная проекция на плоскость  $xy$  описывается матрицей:



$$P_{ortho} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (1.11)$$

Перспективная проекция создает более реалистичное изображение, имитируя работу человеческого глаза, где удаленные объекты кажутся меньше. Данная проекция описывается нелинейным преобразованием, которое в однородных координатах может быть представлено матрицей:

$$P_{persp} = \begin{bmatrix} \frac{2 \cdot n}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2 \cdot n}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2 \cdot f \cdot n}{f-n} \\ 0 & 0 & -1 & 0 \end{bmatrix}, \quad (1.12)$$

где  $l, r, b, t$  – границы усеченной пирамиды видимости по осям  $x$  и  $y$ ,  $n$  и  $f$  – расстояния до ближней и дальней плоскостей отсечения соответственно.

### 1.4.2. Методы удаления невидимых поверхностей

Для корректного отображения взаимного расположения объектов необходимо решить проблему видимости. Наиболее распространенные методы решения этой задачи – Z-буфер и алгоритм художника. Z-буфер использует дополнительный буфер, хранящий глубину для каждого пикселя изображения. Для каждого фрагмента проверяется его глубина  $z_{frag}$  по отношению к текущему значению в буфере  $z_{buffer}$ :

$$\text{if } z_{frag} < z_{buffer}[x, y] \text{ then } \begin{cases} z_{buffer}[x, y] = z_{frag} \\ \text{color}[x, y] = \text{fragment\_color} \end{cases}. \quad (1.13)$$

Алгоритм художника представляет собой метод решения проблемы видимости, основанный на аналогии с техникой рисования художника, который сначала изображает дальний фон, затем средние планы и в конце – ближние объекты. Суть алгоритма заключается в сортировке всех полигонов сцены по убыванию расстояния от точки наблюдения и последующей отрисовке в порядке от наиболее удаленных к ближайшим. Математически это выражается как упорядочение множества полигонов  $P = \{p_1, p_2, \dots, p_m\}$  по ключу  $d_i = \|c - p_i.center\|$ , где  $c$  – позиция камеры, а  $p_i.center$  – центр полигона  $p_i$ . Однако данный алгоритм имеет принципиальные ограничения: он не может корректно обрабатывать случаи взаимного пересечения полигонов или циклических перекрытий, например, когда полигон А перекрывает В, В перекрывает С, а С перекрывает А. Для преодоления этих ограничений применяются дополнительные техники, такие как разбиение полигонов на более мелкие части или использование гибридных подходов.

Альтернативные методы решения проблемы видимости включают алгоритм Варнока и алгоритм Робертса. Алгоритм Варнока основан на рекурсивном разбиении экрана на подобласти до тех пор, пока в каждой области не останется тривиальная конфигурация для определения видимости. Алгоритм работает в объектном пространстве и на каждом шаге рекурсии анализирует текущую область изображения: если в области находится только один полигон, он отрисовывается полностью; если полигонов несколько и они не перекрываются, выбирается ближайший; если конфигурация сложная, область разделяется на четыре подобласти, и процесс повторяется. Данный подход эффективен для сцен с большим количеством однородных областей, но имеет высокую вычислительную сложность для детализированных изображений.

Алгоритм Робертса является одним из первых методов удаления невидимых линий и поверхностей. Он работает на уровне отдельных рёбер каркасной модели и основан на аналитической геометрии. Алгоритм последовательно проверяет каждое ребро объекта на видимость, анализируя его взаимное расположение с другими объектами сцены. Для каждого ребра строится уравнение плоскости, содержащей это ребро, и проверяется, не перекрывается ли оно другими поверхностями. Основное преимущество алгоритма – точность определения видимых контуров, однако его практическое применение ограничено высокой вычислительной сложностью  $O(n^2)$  для  $n$  объектов и сложностью обработки неконвексных моделей.

Для визуализации городской среды метод Z-буфера является предпочтительным, так как обеспечивает корректное отображение взаимного расположения зданий и инфраструктуры без необходимости сложной сортировки геометрии [3].

### 1.4.3. Методы освещения и расчета теней

Для создания реалистичного изображения необходимо моделировать взаимодействие света с поверхностями объектов. В компьютерной графике существует несколько основных подходов к расчету освещения.

Существует несколько классических моделей освещения, отличающихся сложностью вычислений и визуальным качеством. Модель Фонга предполагает интерполяцию нормалей к поверхности в пределах полигона с последующим расчетом освещения для каждого пикселя. Интенсивность освещения в точке рассчитывается по формуле:

$$I = I_a \cdot k_a + I_l \cdot k_d \cdot (\mathbf{N} \cdot \mathbf{L}) + I_l \cdot k_s \cdot (\mathbf{R} \cdot \mathbf{V})^n, \quad (1.14)$$

где  $I_a$ ,  $I_l$  – интенсивности фонового и направленного освещения соответственно,  $k_a$ ,  $k_d$ ,  $k_s$  – коэффициенты фонового, диффузного и зеркального отражения,  $\mathbf{N}$  – нормаль к поверхности,  $\mathbf{L}$  – направление к источнику света,  $\mathbf{R}$  – отраженный вектор,  $\mathbf{V}$  – направление к наблюдателю,  $n$  – степень зеркального отражения [4].

Закраска по Гуро использует интерполяцию цвета между вершинами полигона.

Освещение рассчитывается только в вершинах, а затем цвет интерполируется по поверхности полигона. Этот метод менее ресурсоемкий, чем модель Фонга, но может приводить к артефактам в виде “световых пятен” на гладких поверхностях [5].

Рей-трейсинг представляет собой метод глобального освещения, моделирующий физическое поведение света. Алгоритм трассирует лучи от наблюдателя через каждый пиксель изображения, рассчитывая отражения, преломления и тени. Математически интенсивность пикселя определяется рекурсивным соотношением:

$$I_p = I_{direct} + \sum_{i=1}^n k_{r_i} \cdot I_{reflected_i} + \sum_{j=1}^m k_{t_j} \cdot I_{refracted_j}, \quad (1.15)$$

где  $I_{direct}$  – прямое освещение,  $k_{r_i}$ ,  $k_{t_j}$  – коэффициенты отражения и преломления,  $I_{reflected_i}$ ,  $I_{refracted_j}$  – интенсивности отраженных и преломленных лучей [6].

Для визуализации городской среды методы с интерполяцией освещения не были выбраны, поскольку здания преимущественно имеют явные углы и геометрические особенности, где интерполяция нормалей или цвета не требуется. Рей-трейсинг, несмотря на высокое качество результата, требует значительных вычислительных ресурсов и не подходит для интерактивной визуализации больших городских моделей [7].

Был выбран метод однотонной закраски с расчетом диффузной яркости, поскольку он обеспечивает хороший баланс между вычислительной сложностью и визуальным качеством. Диффузная составляющая зависит от угла падения света на поверхность и описывается законом Ламберта:

$$I_{diffuse} = I_l \cdot k_d \cdot \max(0, \mathbf{N} \cdot \mathbf{L}). \quad (1.16)$$

Для расчета теней используется метод карты теней, который включает два основных шага.

- 1) Рендеринг сцены с точки зрения источника света с сохранением буфера глубины  $z_{light}$ .
- 2) Рендеринг сцены с точки зрения наблюдателя с проверкой видимости каждого фрагмента из точки зрения источника света.

$$\text{if } z_{frag\_to\_light} > z_{light}[u, v] + \epsilon \text{ then fragment in shadow,} \quad (1.17)$$

где  $u, v$  – координаты в текстурном пространстве карты теней,  $\epsilon$  – небольшая константа для борьбы с артефактами [8].

#### 1.4.4. Выбранные методы для визуализации городской среды

Для решения задачи трехмерной визуализации городской среды были выбраны следующие методы:

**Полигональное представление с треугольными полигонами** – данное пред-

ставление обеспечивает оптимальное соотношение между точностью геометрического описания и вычислительной эффективностью. Здания и инфраструктура города могут быть эффективно аппроксимированы треугольными сетками, что позволяет использовать современные графические API для ускорения рендеринга.

**Z-буфер для определения видимости** – метод Z-буфера обеспечивает корректное отображение взаимного расположения объектов без необходимости предварительной сортировки геометрии. Алгоритм работает в два этапа: сначала вычисляется глубина каждого фрагмента, затем производится сравнение с текущим значением в буфере глубины для принятия решения о видимости фрагмента.

**Однотонная закраска с диффузной составляющей** – для создания реалистичного освещения используется модель, учитывающая диффузное отражение света от поверхностей зданий. Интенсивность освещения рассчитывается по формуле:

$$I = (\mathbf{N} \cdot \mathbf{L}) \cdot I_{light}, \quad (1.18)$$

где  $\mathbf{N}$  – нормаль к поверхности,  $\mathbf{L}$  – направление к источнику света,  $I_{light}$  – интенсивность источника света.

**Метод карты теней** – для расчета теней от зданий и других объектов используется алгоритм карты теней. Процесс включает генерацию карты глубины с точки зрения источника света и последующую проверку нахождения фрагментов в тени при рендеринге сцены с точки зрения наблюдателя. Метод обеспечивает реалистичное отображение теней, отбрасываемых зданиями на дороги и другие объекты городской среды. Кроме того, карта теней использует в себе Z-буфер, что увеличивает повторное использование кода.

## Вывод

В данной главе были рассмотрены основные методы представления трехмерных объектов, проецирования их на двумерную плоскость и расчета освещения, математические основы аффинных преобразований для трехмерных моделей, включая перенос, масштабирование и вращение в однородных координатах. Для решения задачи визуализации городской среды были выбраны: полигональное представление с треугольными полигонами, метод Z-буфера для определения видимости, однотонная закраска с расчетом диффузной яркости и метод карты теней для расчета теней. Данный набор методов обеспечивает оптимальное соотношение между вычислительной сложностью и визуальным качеством результата.

## 2 Конструкторская часть

В данном разделе разобраны алгоритмы, а также основные структуры данных.

### 2.1. Схемы алгоритмов

Ниже приведены схемы алгоритмов отрисовки сцены и генерации городской среды.

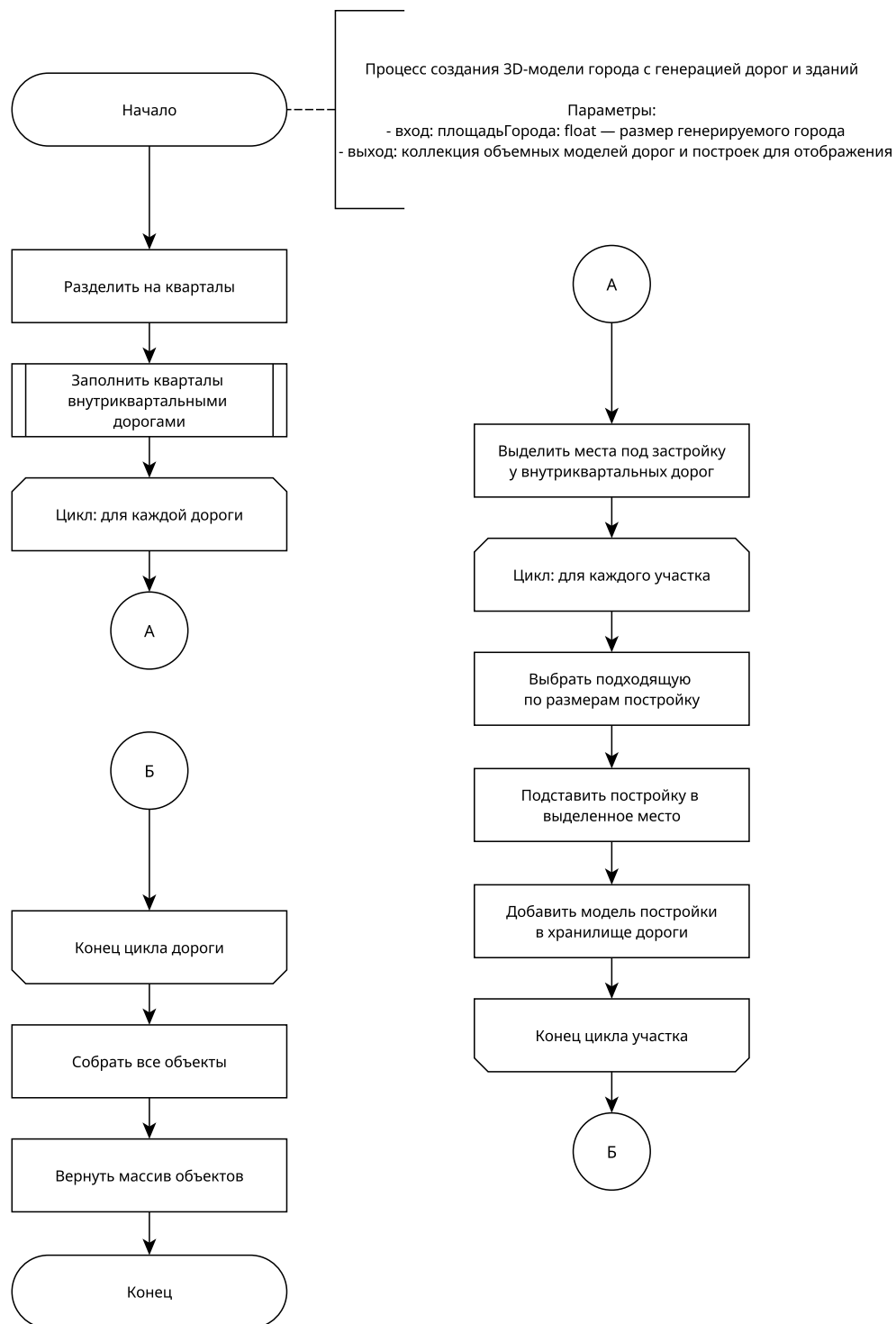


Рисунок 2.1 — Схема алгоритма создания сцены городской среды

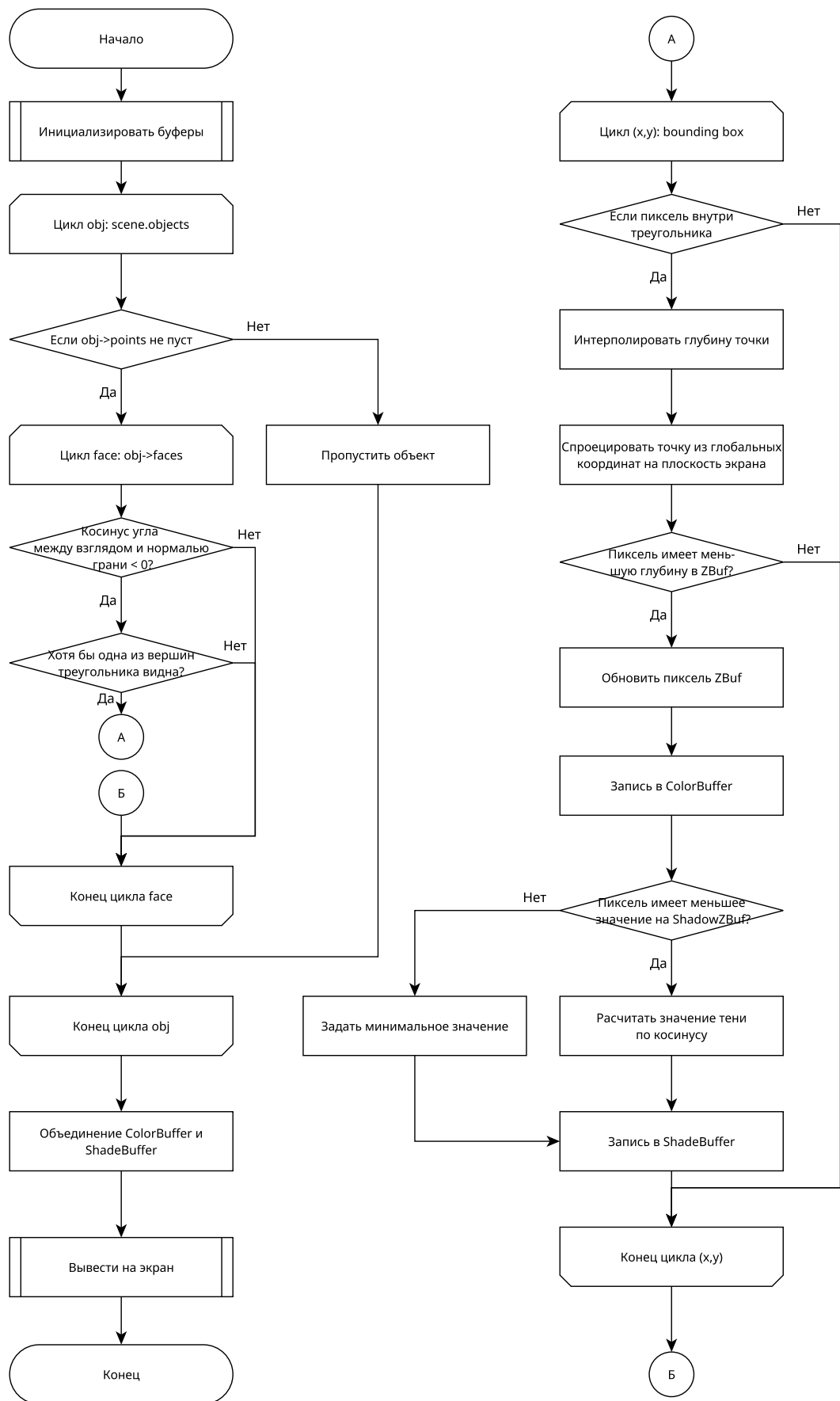


Рисунок 2.2 — Схема алгоритма отрисовки объектов

Распараллеливание вычислений производится при помощи обработки пикселей при растеризации в различных потоках.

## 2.2. Используемые структуры данных

Для реализации программного обеспечения были использованы следующие структуры данных.

### Камера

Камера описывает точку зрения, с которой производится отрисовка сцены. Структура содержит следующие поля:

- трехмерный вектор позиции камеры в пространстве;
- целевая точка, на которую направлена камера;
- вектор вверх.

### Графический объект

Графический объект представляет собой базовую структуру для всех объектов, которые могут быть отрисованы в сцене. Структура содержит:

- массив точек – список вершин, составляющих геометрию объекта;
- массив граней – список граней, определяющих полигональную сетку объекта.

### Грань

Грань описывает грань полигональной сетки и содержит:

- три индекса вершин, образующих треугольную грань;
- нормальный вектор грани, используемый для расчёта освещения;
- цвет грани.

### Сцена

Сцена представляет собой контейнер для всех объектов, находящихся в виртуальном трехмерном пространстве. Структура включает:

- массив графических объектов;
- ссылку на активную камеру;
- массив источников света.

### Вывод

В конструкторской части представлены схемы алгоритмов генерации городской среды и отрисовки объектов. Описаны ключевые структуры данных, используемые в про-

граммной реализации: камера, графический объект, грань и сцена. Определен подход к распараллеливанию вычислений путём обработки пикселей в различных потоках при rasterization.



## 3 Технологическая часть

В данном разделе производится выбор технологий программной реализации, а также приводятся диаграммы классов программного решения и приводятся примеры работы программы.

### 3.1. Выбор технологий для программной реализации

Программная реализация разработана с использованием метода рекурсивного дизайна. Данный подход предполагает поэтапное расширение функциональности, начиная с минимально работоспособного ядра [11]. На каждом этапе система остаётся в рабочем состоянии, что позволяет проводить непрерывное тестирование [12].

В качестве языка программирования выбран C++ стандарта C++20 [13]. Стандарт C++20 предоставляет современные возможности языка, включая концепты и статический полиморфизм для повышения скорости работы. Статическая типизация и прямое управление памятью обеспечивают оптимальное использование вычислительных ресурсов.

Для создания графического интерфейса использован фреймворк Qt версии 6.5 [14]. Фреймворк Qt предоставляет инструменты для разработки графических приложений, включая систему виджетов. Архитектура Qt, основанная на модели MVC, обеспечивает разделение данных, представления и логики управления.

Средой разработки выбран Visual Studio Code версии 1.106 [15]. Среда обеспечивает отладку программного кода с поддержкой точек останова и пошагового выполнения.

Для распараллеливания вычислений при растеризации используется библиотека OMP [16], при использовании директивы `#pragma omp parallel for collapse(2)`, для цикла, перебирающего координаты  $x$  и  $y$  пространства буфера.

### 3.2. Структура классов

UML-диаграммы классов для домена отрисовки и генерации городской среды представлены в приложении А. Домены пересекаются только в момент передачи объемных объектов функцией `CityMap::exportToScene()`. В рамках реализации буфер теней использует ортогональную проекцию, так как в реальном мире расстояние до Солнца предельно велико, по сравнению с масштабами города. Рендер поддерживает наличие более одного источника освещения, в том числе точечных, свет от которых распространяется аналогично перспективной проекции.

### 3.3. Создание городской среды

На рисунках 3.1 и 3.2 представлена UML-диаграмма последовательности, показывающая взаимодействие классов при генерации городской среды.

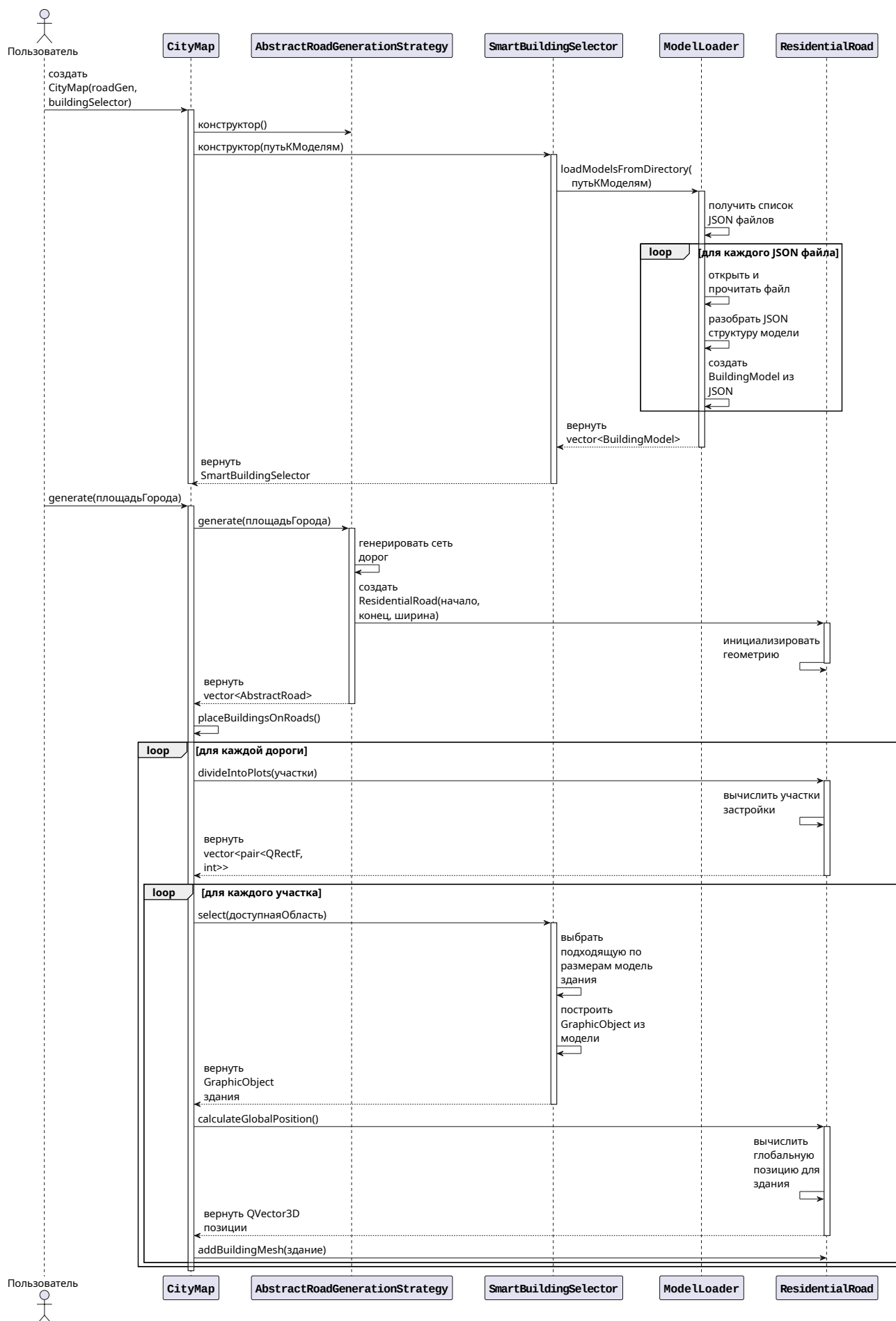


Рисунок 3.1 — UML-диаграмма последовательности создания городской среды, ч.1

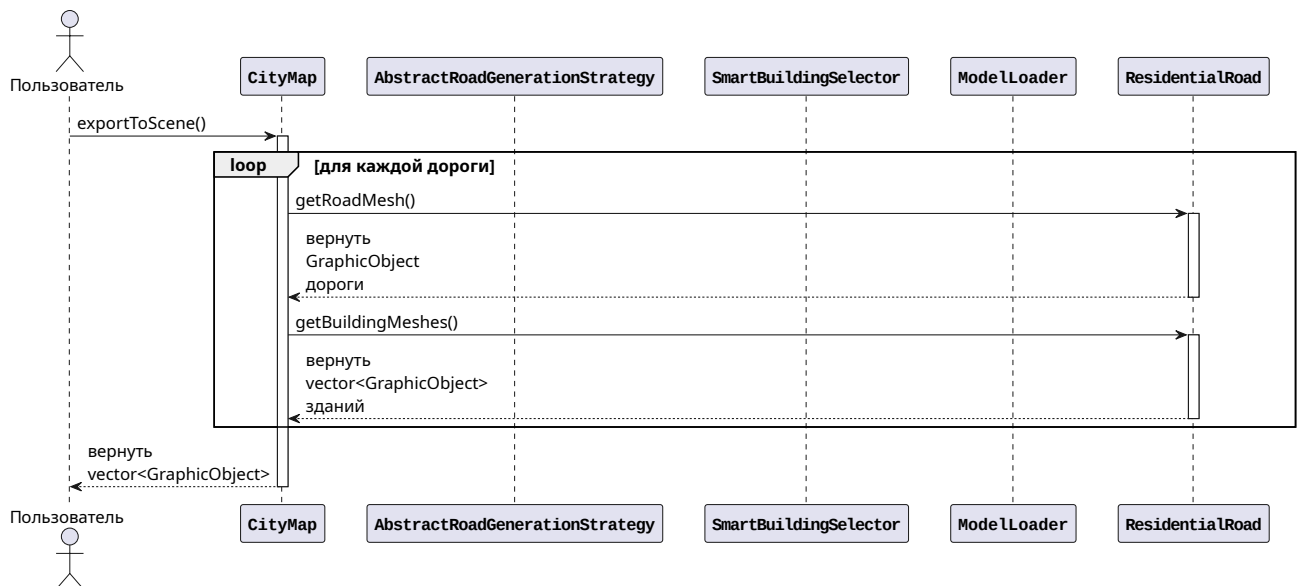


Рисунок 3.2 — UML-диаграмма последовательности создания городской среды, ч.2

### 3.4. Пример работы программы

Интерфейс программы предполагает настройку направления и оттенка света, возможность задания параметров генерации города, а также настройку качества отображения и угла обзора. Перемещение камеры достигается за счёт нажатия клавиш W, A, S, D, а также клавиши Shift для ускорения полета. Вращение камерой достигается за счёт передвигания курсора по изображению, при зажатой левой клавише мыши.

На рисунках 3.3 и 3.4 представлены примеры работы программы.

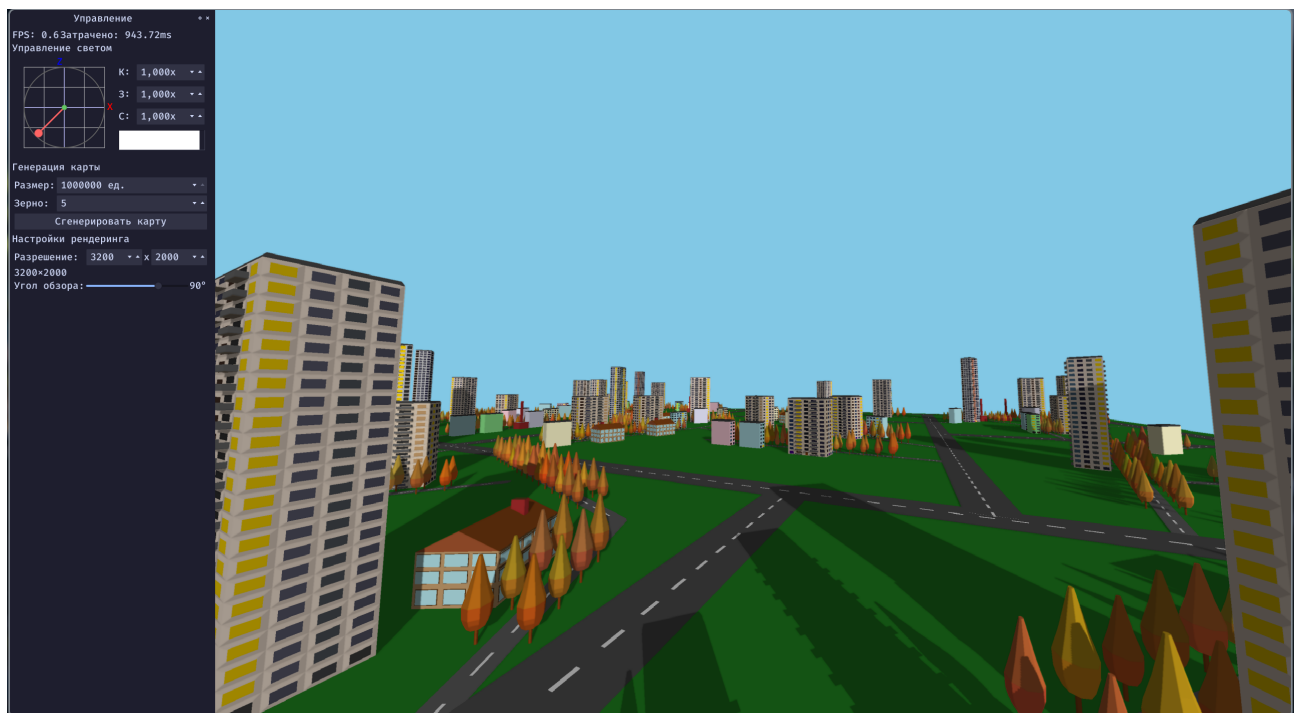


Рисунок 3.3 — Пример работы программы

Интерфейс может быть представлен как вкладкой, так и отдельным плавающим окном, как на примере 3.4.



Рисунок 3.4 — Пример работы программы

## Вывод

В технологической части определены технологии программной реализации: C++20, Qt 6.5 и Visual Studio Code. Представлены UML-диаграммы классов для доменов генерации городской среды и отрисовки, демонстрирующие модульную архитектуру приложения. Описана реализация буфера теней с ортогональной проекцией для солнечного освещения и поддержкой множественных источников света. Приведена диаграмма последовательности процесса генерации городской среды и примеры работы программы с различными конфигурациями городов и освещения.

## 4 Исследовательская часть

В данном разделе проводится исследование зависимости времени отрисовки одного кадра от разрешения квадратного изображения с длиной стороны  $N$  пикселей. Замеры времени будут проводиться на одной и той же сцене, позиция камеры охватывает весь город. В качестве значения будет использоваться усреднённое время отрисовки за 50 кадров.

Исследования производились на машине со следующими характеристиками:

- операционная система — Arch Linux based on Linux 6.17.8 [9];
- процессор — AMD Ryzen 7 8750H (16) [10];
- оперативная память — DDR5 6400МГц 24 ГБ.

Во время замеров ноутбук был подключен к питанию, а также был активирован производительный режим.

### 4.1. Результаты замеров

При тестировании использовался город квадратной формы. Условные единицы, используемые как мера длины в программе, примерно равны метру.

Таблица 4.1 — Время отрисовки квадратного кадра в зависимости от размера стороны

| Размер стороны квадрата | Время отрисовки (мс)        | Время отрисовки (мс)      |
|-------------------------|-----------------------------|---------------------------|
|                         | сторона города 1000000 у.е. | сторона города 40000 у.е. |
| 100                     | 415                         | 14                        |
| 200                     | 420                         | 15                        |
| 300                     | 418                         | 17                        |
| 500                     | 442                         | 27                        |
| 1000                    | 460                         | 56                        |
| 1500                    | 536                         | 87                        |
| 2000                    | 570                         | 127                       |
| 2500                    | 631                         | 190                       |
| 3000                    | 696                         | 276                       |
| 4000                    | 907                         | 385                       |
| 5000                    | 1175                        | 595                       |
| 6000                    | 1550                        | 827                       |
| 7000                    | 1932                        | 1161                      |
| 8000                    | 2253                        | 1451                      |
| 9000                    | 2872                        | 1832                      |
| 10000                   | 3305                        | 2495                      |

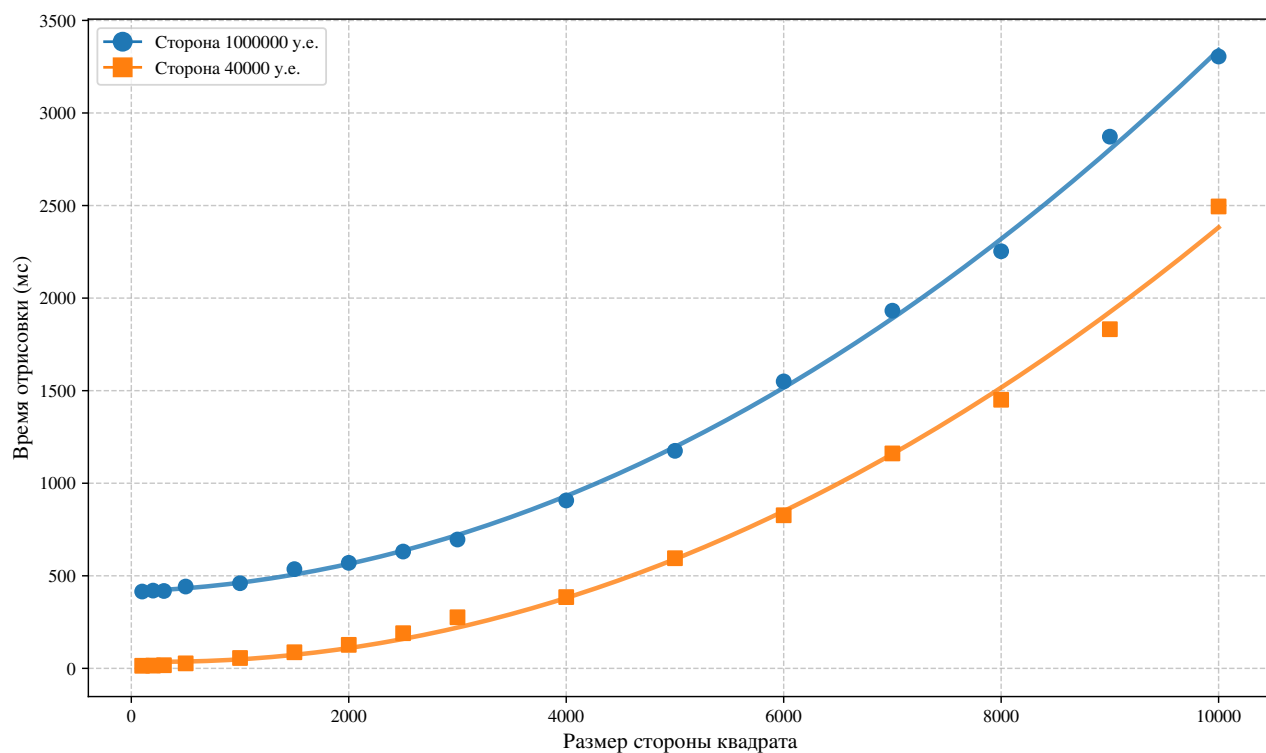


Рисунок 4.1 — Зависимость времени отрисовки квадратного изображения от размера стороны

Линия графика показывает квадратичную аппроксимацию по точкам, полученным в результате проведения замеров.

Проведенные исследования показывают, что время отрисовки имеет квадратичную зависимость от стороны квадратного изображения, однако, при городе размером в 1000000 есть увеличенное константное время, обусловленное перебором большого числа полигонов в процессе отображения.

## Вывод

В результате замеров выяснилось, что при квадратном изображении время отрисовки кадра квадратично зависит от длины стороны. При наличии большого числа полигонов существует константная задержка, обусловленная перебором всех полигонов.

# ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы разработано программное решение для генерации и трёхмерной визуализации городской среды, полностью удовлетворяющее поставленному заданию. Все задачи, сформулированные во введении, успешно реализованы:

- создан алгоритм генерации транспортной сети с тремя и более перекрёстками, гарантирующий отсутствие изолированных участков;
- созданы объемные модели зданий пяти архитектурных типов с минимальной детализацией 20 полигонов на объект и алгоритм их размещения вдоль дорог без взаимных пересечений;
- обеспечена воспроизводимость генерации за счёт фиксации начального значения генератора псевдослучайных чисел;
- реализовано интерактивное управление камерой, а именно, перемещение, вращение, масштабирование;
- интегрирован алгоритм карты теней для расчёта динамического освещения с возможностью настройки источников света.

При выполнении работы сделаны следующие заключения:

- в аналитической части разобраны математические модели аффинных преобразований, определены методы, используемые в программе, а именно Z-буфер для определения перекрытия полигонов, диффузной закраски и карты теней для решения задачи теней и освещения;
- в конструкторской части, разработан алгоритм генерации городской среды и отрисовки кадра, а также предложен подход к распараллеливанию растеризации;
- реализация на базе C++20 и Qt 6.5 позволила создать модульную архитектуру приложения с чётким разделением логики генерации, рендеринга и управления пользовательским интерфейсом;
- исследования показали квадратичную зависимость времени отрисовки от разрешения кадра и выявили константную задержку при обработке большого числа полигонов, что обусловлено необходимостью перебора всех объектов сцены.

Таким образом, цель работы — создание программного комплекса для автоматической генерации и интерактивной визуализации городской среды с использованием современных методов рендеринга — достигнута.

# СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Kaji, S., Ochiai, H. A Concise Parametrization of Affine Transformation / SIAM Journal on Imaging Sciences. — 2016. — Vol. 9, No. 3. — Pp. 1355–1372.
2. Rogers, D. F. Mathematical Elements for Computer Graphics / D. F. Rogers, J. A. Adams. — New York: McGraw-Hill, 1990. — Pp. 15–87.
3. Rogers, D. F. Procedural Elements for Computer Graphics / D. F. Rogers. — New York: McGraw-Hill, 1985. — Pp. 123–215.
4. Phong, Bui Tuong. Illumination for computer generated pictures / Communications of the ACM. — 1975. — Vol.18, No.6. — Pp. 311–317.
5. Gouraud, Henri. Continuous shading of curved surfaces / IEEE Transactions on computers. — 1971. — Vol.20, No.6. — Pp. 623–629.
6. Whitted, Turner. An improved illumination model for shaded display / Communications of the ACM. — 1980. — Vol.23, No.6. — Pp. 343–349.
7. Akenine-Möller, Tomas. Real-time rendering / AK Peters/CRC Press. — 2018. — 1198 p.
8. Williams, Lance. Casting curved shadows on curved surfaces / ACM SIGGRAPH Computer Graphics. — 1978. — Vol.12, No.3. — Pp. 270–274.
9. ArchWiki — Arch Linux. Главная страница. [Электронный ресурс]. Режим доступа: [https://wiki.archlinux.org/title/Main\\_page](https://wiki.archlinux.org/title/Main_page) (дата обращения: 9 декабря 2025 г.).
10. AMD Ryzen™ 7 8745HX Mobile Processor. [Электронный ресурс]. — Режим доступа: <https://www.amd.com/en/products/processors/laptop/ryzen/8000-series/amd-ryzen-7-8745hx.html> (дата обращения: 9 декабря 2025 г.).
11. Fowler, M. Refactoring: Improving the Design of Existing Code / Addison-Wesley Professional. — 2018. — Pp. 45–98.
12. Booch, G., Rumbaugh, J., Jacobson, I. The Unified Modeling Language User Guide / Addison-Wesley Professional. — 2005. — Pp. 132–156.
13. ISO/IEC 14882:2020. Programming languages — C++ / International Organization for Standardization. — 2020. — 1853 p.
14. The Qt Company. Qt 6.5 Documentation / Qt Documentation. — 2023. — Режим доступа: <https://doc.qt.io/qt-6/> (дата обращения: 9 декабря 2025 г.).



15. Microsoft Corporation. Debugging in Visual Studio Code / VS Code Docs. — 2023. — Режим доступа: <https://code.visualstudio.com/docs/> (дата обращения: 9 декабря 2025 г.).
16. OpenMP Architecture Review Board. OpenMP Application Program Interface Specification Version 6.0 / OpenMP.org. — 2024. — Режим доступа: <https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-6-0.pdf> (дата обращения: 9 декабря 2025 г.).

# ПРИЛОЖЕНИЕ А

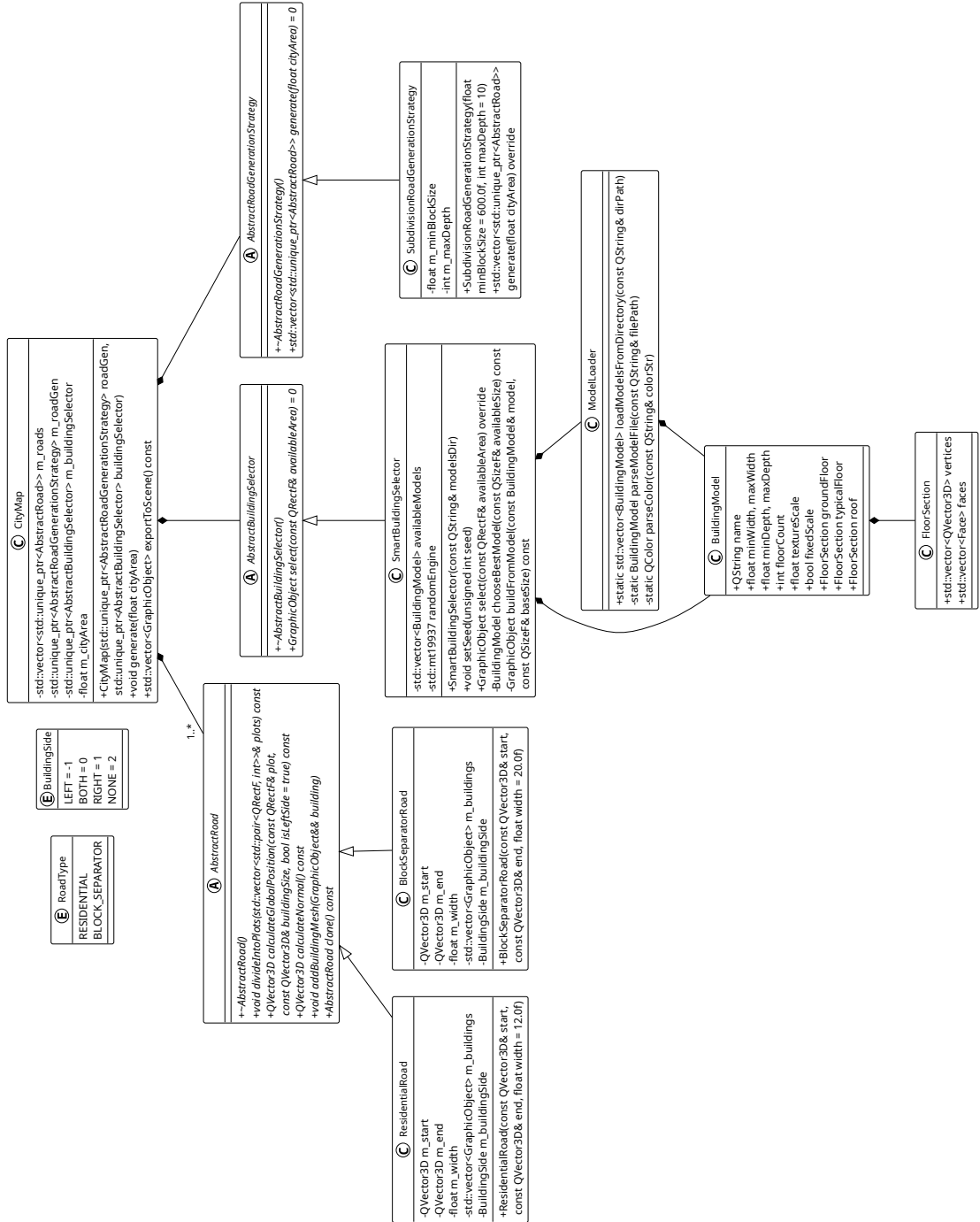


Рисунок 1 — Диаграмма классов домена генерации городской среды

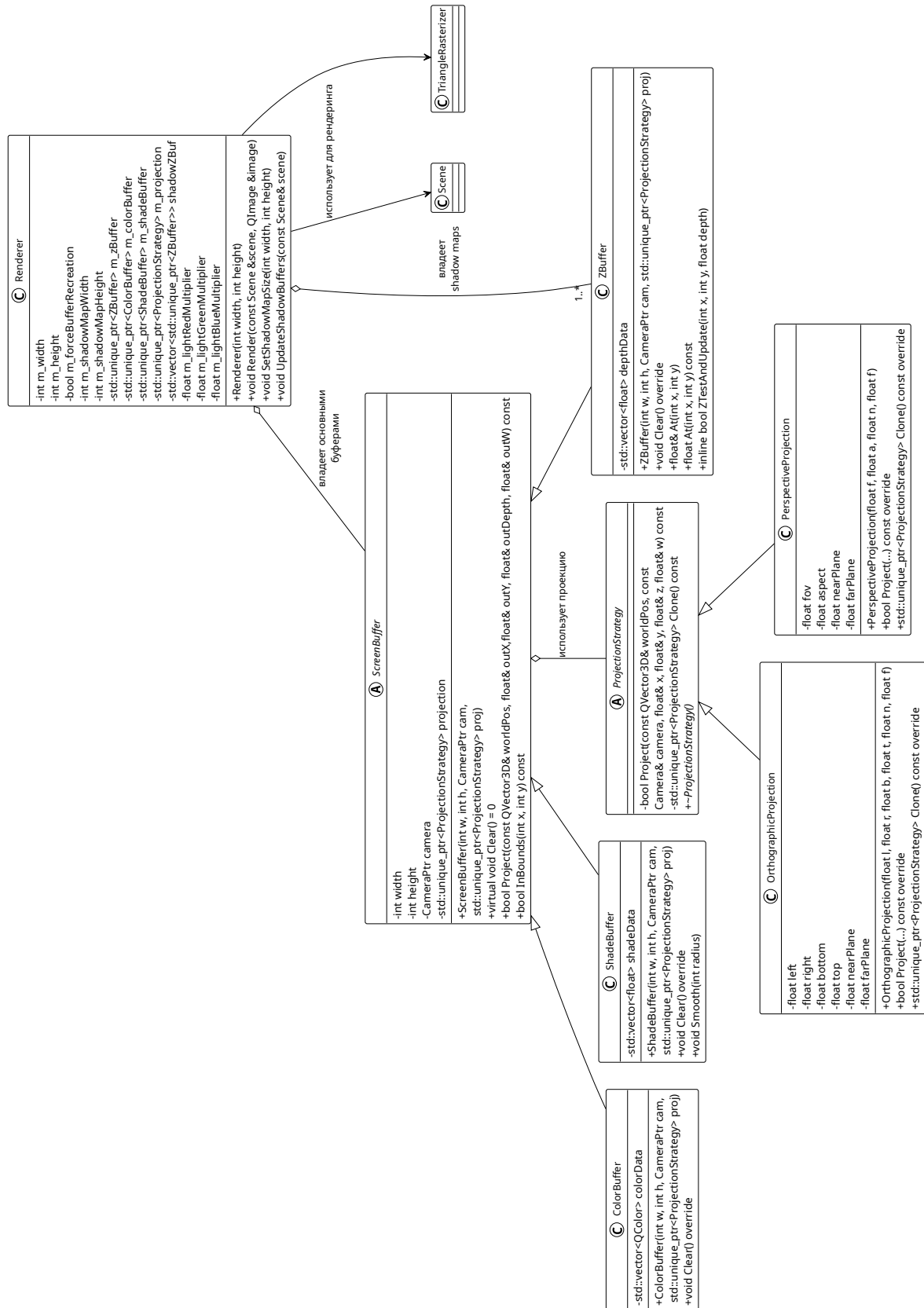


Рисунок 2 — Диаграмма классов домена отрисовки, ч. 1

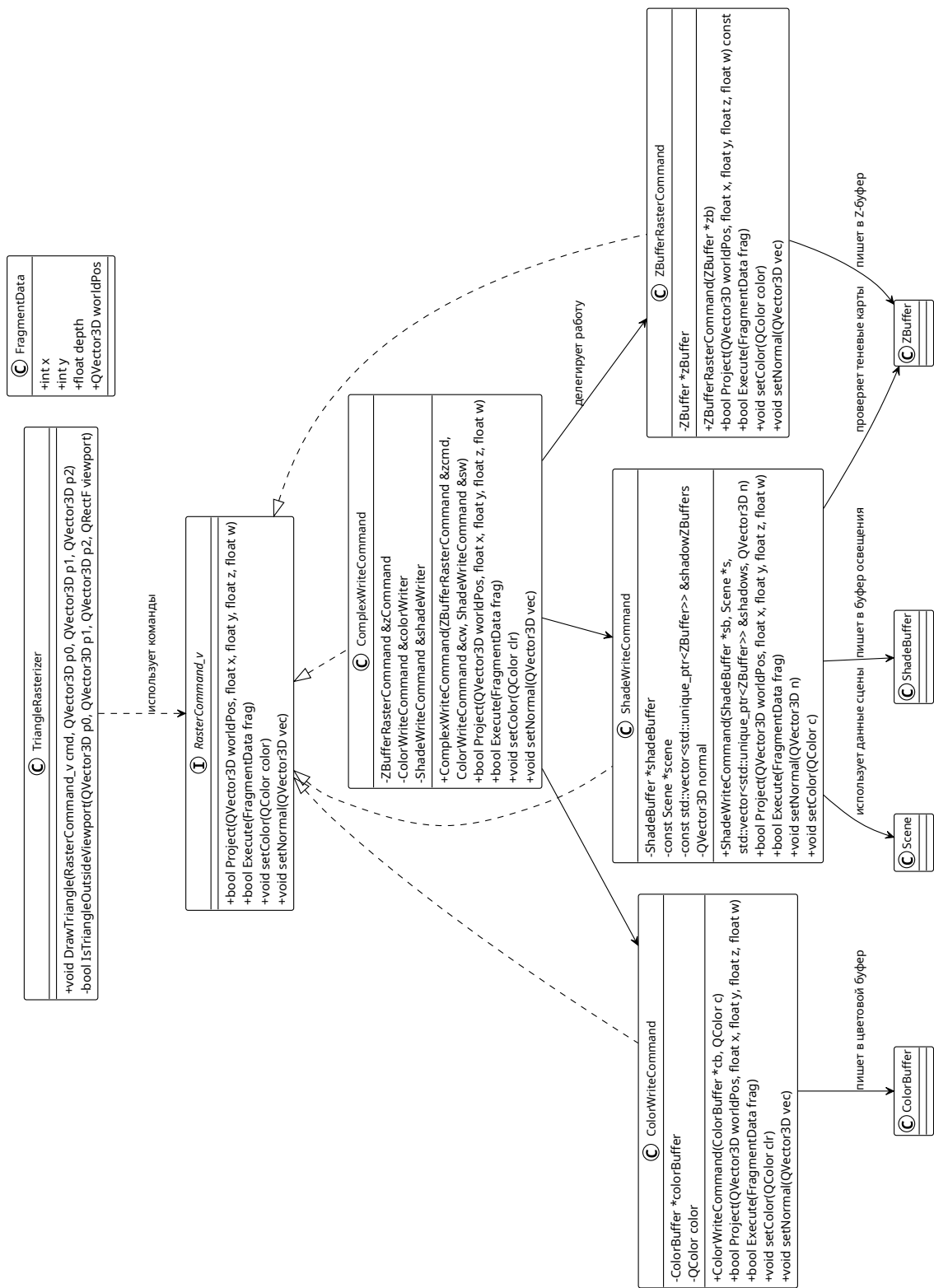


Рисунок 3 — Диаграмма классов домена отрисовки, ч. 2

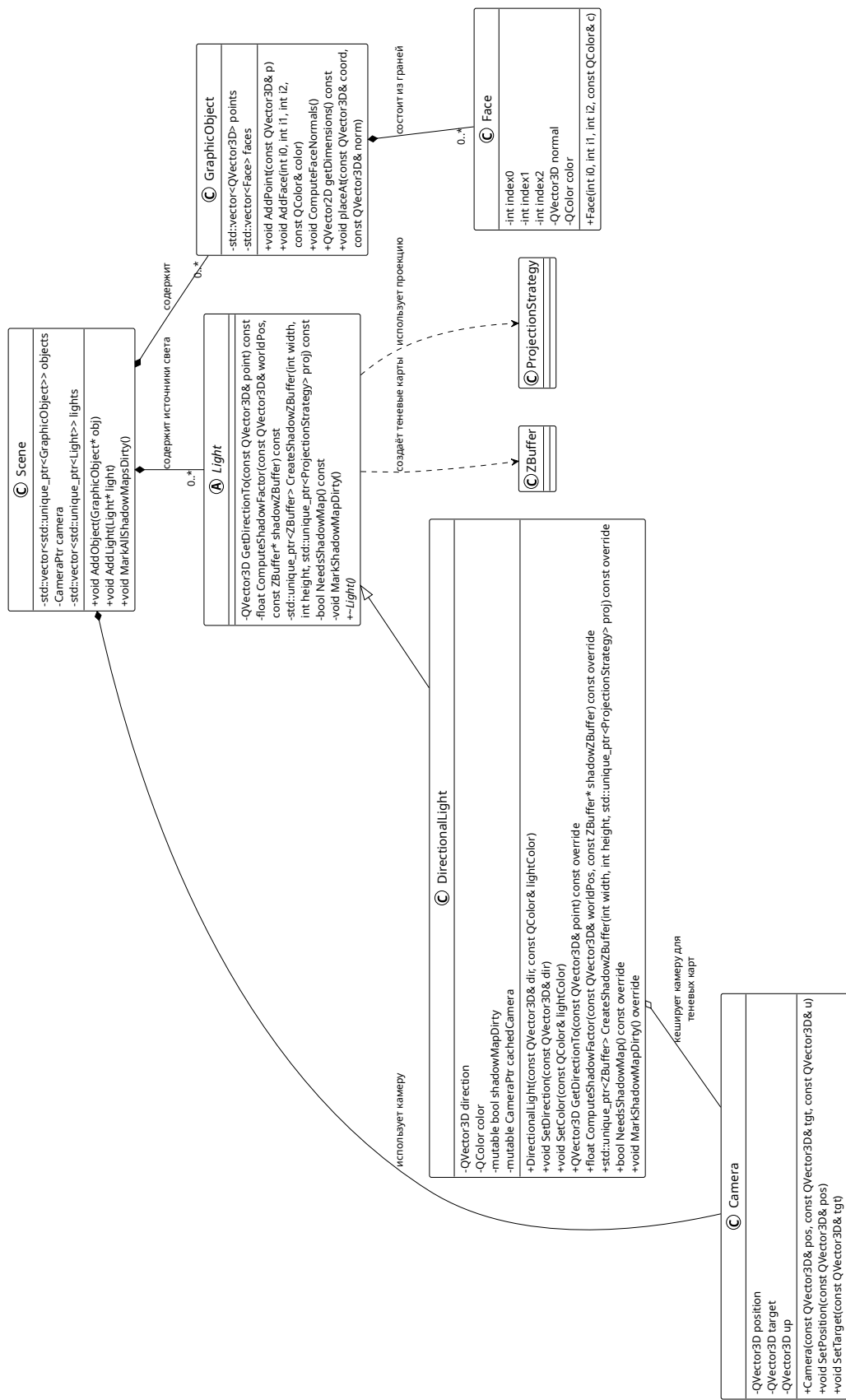


Рисунок 4 — Диаграмма классов домена отрисовки, ч. 3

# ПРИЛОЖЕНИЕ Б

К РПЗ работы прилагаются следующие материалы:

1. Презентация к курсовой работе, состоящая из 16 слайдов.
2. USB-флеш-накопитель с исходными кодами, электронными версиями РПЗ и презентации. Структура файлов на накопителе:

-rpz/

|

|—report.pdf – Итоговая РПЗ

|—presentation.pdf – Презентация

-code/ – исходный код программы